To-Do List (Basic) – Documentation

In this guide, you will learn how to create a simple word guessing game using Python. This project will help students understand fundamental programming concepts such as loops, conditionals, and user input handling while enhancing their logical thinking and problem-solving skills.

Overview

The **To-Do List** project is a **simple command-line-based application** that allows users to **add, view, mark, and delete tasks** using Python.

- **User can add tasks**
- **User can view pending tasks**
- **User can remove tasks**
- **User can mark tasks as completed**
- **Uses a simple loop for continuous interaction**

Why Build This Project?

This project helps students learn:

- How to use dictionaries to store word-hint pairs.
- Implementing loops to check user input.
- Handling string operations to display guessed words.
- Managing user attempts using conditional statements.

Prerequisites

Before starting this project, ensure you are familiar with:

- Basic Python syntax
- Loops and conditionals
- String manipulation
- Dictionaries and lists in Python

Step-by-Step Implementation

Step 1: Setting Up the Project

First, create a new project folder and add the following file:

```
/to-do-list
│── todo.py  (Main Python script)
```

This file will contain the Python logic for the to-do list application.

Step 2: Writing the Python Code

Open **todo.py** and add the following code:

The **Python script** contains all the logic to manage the to-do list.

(a) Importing Modules

```
import os
```

- The **os module** is used to clear the screen for better user experience.

(b) Defining the To-Do List & Functions

```
# List to store tasks
tasks = []
```

- **tasks is a list** that holds all the tasks added by the user.

(c) Displaying the Menu

```
def show_menu():
    """Displays the To-Do List menu."""
    print("\nTo-Do List Menu:")
    print("1. View Tasks")
    print("2. Add Task")
    print("3. Remove Task")
    print("4. Mark Task as Done")
    print("5. Exit")
```

- This function, show_menu, prints a simple text-based menu for a To-Do List application, displaying five options:
  viewing tasks, adding a task, removing a task, marking a task as done, and exiting the application.

(d) Viewing Tasks

```
def view_tasks():
    """Displays the list of tasks."""
    if not tasks:
        print("No tasks available.")
    else:
        print("\nYour Tasks:")
        for index, task in enumerate(tasks, start=1):
            print(f"{index}. {task}")
```

- If the **tasks list is empty**, it displays "No tasks available."
- This function displays the list of tasks. If the list is empty, it prints "No tasks available." Otherwise, it prints each task with a numbered index starting from 1.

(e) Adding a Task

```
def add_task():
    """Adds a new task to the list."""
    task = input("Enter a new task: ")
    tasks.append(task)
    print(f"Task '{task}' added successfully.")
```

- This function, add_task, prompts the user to enter a new task, adds the input to the tasks list, and then prints a confirmation message indicating the task was added successfully.

(f) Removing a Task

```
def remove_task():
    """Removes a task based on user input."""
    view_tasks()
    try:
        task_number = int(input("Enter task number to remove: ")) - 1
        if 0 <= task_number < len(tasks):
            removed_task = tasks.pop(task_number)
            print(f"Task '{removed_task}' removed successfully.")
        else:
            print("Invalid task number.")
    except ValueError:
        print("Please enter a valid number.")
```

- This function, remove_task, allows the user to delete a task from the list by entering its corresponding task number. It first displays the current tasks (view_tasks()), then prompts the user for the task number to remove. If the input is a valid number and within the range of existing tasks, it removes the task and prints a success message. Otherwise, it displays an error message.
- **Checks if the number is valid**, then removes the task using .pop().

(g) Marking a Task as Done

```
def mark_task_done():
    """Marks a task as completed."""
    view_tasks()
    try:
        task_number = int(input("Enter task number to mark as done: ")) - 1
        if 0 <= task_number < len(tasks):
            tasks[task_number] += " ✔ (Done)"
            print("Task marked as completed.")
        else:
            print("Invalid task number.")
    except ValueError:
        print("Please enter a valid number.")
```

- Adds "✔ (Done)" to the selected task.
- This code defines a function mark_task_done() that marks a task as completed by:
- Displaying the current tasks (view_tasks())
- Asking the user to input the task number to mark as done
- Validating the input (checking if it's a valid number and within the range of existing tasks)
- If valid, appending " ✔ (Done)" to the corresponding task in the tasks list
- Printing a success message or an error message if the input is invalid.

(h) Main Program Loop

```
# Main program loop
while True:
    show_menu()
    choice = input("Enter your choice: ")

    if choice == "1":
```

```
        view_tasks()
    elif choice == "2":
        add_task()
    elif choice == "3":
        remove_task()
    elif choice == "4":
        mark_task_done()
    elif choice == "5":
        print("Exiting... Goodbye!")
        break
    else:
        print("Invalid choice. Please enter a number from 1 to 5.")
```

- **Continuously runs** until the user selects **Exit (5)**.
- **Handles invalid choices** by displaying an error message.

How to Run the Project

1. **Install Python** (if not installed).
2. Open the terminal or command prompt.
3. Navigate to the project folder.
4. Run the script using:
   python todo.py
5. Follow the on-screen menu to add, view, remove, and mark tasks.

What You Learned

- **Simple & Interactive** – Easy to use with a clear menu
- **Task Management** – Supports adding, deleting, and marking tasks.
- **Loop-Based Execution** – Runs until the user exits.
- **Error Handling** – Prevents crashes from invalid inputs.

Conclusion

This documentation explains how the **Basic To-Do List (Python)** project works, covering its **functions, logic, and user interactions**. This project **helps beginners understand Python fundamentals like lists, loops, and functions**.